

is expected to support, insert, delete and find operations, please compare the following data structures as potential candidates in terms of time complexity:

- Unsorted array
- Sorted array
- Unsorted singly linked list
- Sorted singly linked list
- Unsorted doubly linked list
- Sorted doubly linked list
- Binary search tree
- Hash table
- Binary heap

10. We are all familiar with merge-sort. Given an array of N integer elements, let's break up the array into two halves, sort each half separately and then merge the two sorted halves. This is a classic example of the 'divide and conquer' approach, by which the problem is broken down into several sub-problems. Each is solved recursively, and then you combine the solutions for the sub-problems to solve the original problem. Is it always possible to design a 'divide and conquer' algorithm to all problems? If not, what characteristics should a problem exhibit to be amenable to this approach? Can you give an example of a problem which is not amenable to the 'divide and conquer' approach and explain why it is not suitable?

11. You are given a set 'A' containing N integer elements and asked to solve the problem of finding the minimum in a given range of A. For instance, if A is {8, 2, 9, 5, 1, 4, 11, 3, 14}, the range minimum for A[1...5] is given by the element A[5], since A[5] is 1 and that is the minimum element in the range A[1...5]. Can you come up with an $O(N)$ algorithm for solving the range minimum query if you are allowed to preprocess the set A?

12. Let A be a multi-set of integers consisting of N numbers. You are allowed to pick up two integers at a time. If both are even or both are odd, you discard both and insert an even integer. If not, you discard the even integer. Given that there are an even number of odd integers initially in A, can you tell whether the last integer will be even or odd? What is the reasoning behind your answer?

13. You have 100 doors in a row, all of which are initially closed. You make 100 passes over these doors starting with the first door. The first time when you make the pass, you visit every door and toggle the door (i.e., if the door is open, you close it. And if it is closed, you open it). During the second pass, you visit every second door (doors numbered 2, 4, 6, 8...). During the third pass, you visit only every third door (doors marked 3, 6, 9, 12...). You repeat these passes until you finish all the 100 passes. Now can you determine what state each door is in, after the 100th pass? Which doors are open and which are closed?

14. The lowest common ancestor (LCA) of two nodes 'u' and 'v' in a binary search tree is defined as node A, which has both 'u' and 'v' as its descendants (note that a node can be its own

descendant). Write an algorithm to find the lowest common ancestor of a binary search tree. Instead of writing a new algorithm for finding the LCA, if you are given the in-order traversal of the binary search tree along with the nodes 'u' and 'v' for which the LCA needs to be found, can you use the in-order traversal information to determine the LCA? Does this work in all cases? If so, explain why. If not, give a counter-example.

15. You are given an undirected graph $G(V, E)$ where V represents the set of vertices and E represents the set of edges, and a weight function $c(u, v)$, which associates a non-negative weight with the edges of the graph. A minimum weighted cycle in the graph is defined as the cycle, whose sum of edge weights is the minimum over all cycles in the graph. A maximum weighted cycle is one whose sum of edge weights is the maximum over all the cycles in the graph.

- (i) Can you come up with an algorithm for finding the minimum weighted cycle? What is the complexity of your algorithm?
- (ii) Can you come up with an algorithm for finding the maximum weighted cycle? What is the complexity of your algorithm? Can you come up with a polynomial time algorithm for this problem? If not, why not?

My 'must-read book' for this month

This month's book suggestion comes from one of our readers, Shruti, and her recommendation is very appropriate to this month's column. She recommends an excellent resource for algorithmic problems—the website containing a discussion on algorithms from Jeff Erickson, available at <http://www.cs.uiuc.edu/~jeffe/teaching/algorithms/>. The link has pointers to lecture notes as well as additional exercises in algorithms and data structures. Thank you, Shruti, for sharing this link.

If you have a favourite programming book/article that you think is a must-read for every programmer, please do send me a note with the book's name, and a short write-up on why you think it is useful so I can mention it in the column. This would help many readers who want to improve their software skills.

If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming! 

By: Sandya Mannarswamy

The author is an expert in systems software and is currently working with Hewlett Packard India Ltd. Her interests include compilers, multi-core and storage systems. If you are preparing for systems software interviews, you may find it useful to visit Sandya's LinkedIn group 'Computer Science Interview Training India' at [http://www.linkedin.com/groups?home=HYPERLINK "http://www.linkedin.com/groups?home=&gid=2339182"&HYPERLINK "http://www.linkedin.com/groups?home=&gid=2339182"&gid=2339182](http://www.linkedin.com/groups?home=HYPERLINK%20http://www.linkedin.com/groups?home=&gid=2339182%20&HYPERLINK%20http://www.linkedin.com/groups?home=&gid=2339182%20gid=2339182)